

Installing and testing 15LSS_sample_gas on the IOC host

For the person installing and commissioning the IOC. Placeholders: `<support>` is the synApps support directory on the Linux soft-IOC host, `<ioc-host>` that host, `<user>` the IOC account. The IOC serves `15IDC:SampleGas:` and drives the Alicat `15IDC:Alicat1:` from the O2 reading `15IDC:D1Dmm_calc`.

Read this first. A normal start (`start_ioc`) is *live*: if helium is already flowing (O2 below 10 % and the Alicat setpoint above 0) it takes over the Alicat within a couple of seconds. The first start below is therefore a shadow start, which writes nothing. Stop the old Phoebus timer script before you switch to live: two writers fight over the setpoint.

1. Get the code onto the IOC host

The IOC is public at <https://github.com/Dave-J-W/helium-flow-fb> (branch `main`). The EPICS top is the repo's `ioc/lssSampleGas`; it goes to `<support>/ChemMat/lssSampleGas`.

A. From GitHub (if the host reaches github.com):

```
git clone https://github.com/Dave-J-W/helium-flow-fb.git ~/helium-flow-fb
git -C ~/helium-flow-fb log -1 --oneline          # note the version you install
mkdir -p <support>/ChemMat
rsync -a ~/helium-flow-fb/ioc/lssSampleGas <support>/ChemMat/
```

B. Without internet on the host: make a package on the PC with the repo (Git Bash, in the repo worktree; the committed state is what gets packaged), then copy it over (e.g. `scp`):

```
git -c core.autocrlf=false archive --format=tar.gz \
  --prefix=lssSampleGas/ -o ~/lssSampleGas.tar.gz HEAD:ioc/lssSampleGas
git -c core.autocrlf=false archive --format=zip \
  -o ~/sampleGas_screens.zip HEAD:ioc/screens
```

and on the host:

```
cd <support>/ChemMat && tar xzf ~/lssSampleGas.tar.gz
```

Keep `-c core.autocrlf=false`. Archiving a subdirectory skips the repo's `.gitattributes`, so on a Windows checkout `git archive` would otherwise write CRLF line endings. A trailing `\r` (in `#!/bin/bash\r` or `EPICS_BASE=...\r`) breaks the start scripts and the build on Linux. A clone on the host (A) has no such problem.

The screens are the repo's `ioc/screens/*.bob` (or `sampleGas_screens.zip`): copy them to wherever your Phoebus displays live (section 5).

2. Configure (on the IOC host)

```
cd <support>/ChemMat/lssSampleGas
cp configure/RELEASE.local.production.example configure/RELEASE.local
nano configure/RELEASE.local          # set SUPPORT=<support>
```

`configure/RELEASE` already names the module versions: seq R2-2-9 (as `sequencer-mirror-R2-2-9`), std R3-6-4, calc R3-7-5, asyn R4-44-2, autosave R5-11, sscan R2-11-6, and base `/usr/local/epics/base`. Check they exist:

```
cd <support>
ls -d sequencer-mirror-R2-2-9 std-R3-6-4 calc-R3-7-5 \
    asyn-R4-44-2 autosave-R5-11 sscan-R2-11-6
```

3. Build

```
make 2>&1 | tee build.log
ls -l bin/linux-x86_64/lssSampleGas iocBoot/iocLSS_sample_gas/envPaths
```

This is the first build with gcc on Linux (the bench is MinGW). Compiler warnings are expected; errors are not. If `make` stops with "Definition of ... conflicts with", the modules were built against different versions than `configure/RELEASE` says. Either correct `RELEASE`, or put `CHECK_RELEASE = WARN` into `configure/CONFIG_SITE.local` if you know the mix is fine.

Then run the unit tests on the host. They take a few seconds and catch a C library that formats or rounds differently:

```
lssSampleGasApp/src/0.linux-x86_64/sgUnitTest | tail -1      # PASS: 38/38
lssSampleGasApp/src/0.linux-x86_64/sgIocTest  | tail -1      # PASS: 24/24
lssSampleGasApp/src/0.linux-x86_64/sgFmtTest  | tail -1
```

4. First start: shadow, in a terminal

Only one `lssSampleGas` may run at a time: check it isn't already running under `procServ`. Then:

```
cd <support>/ChemMat/lssSampleGas/iocBoot/iocLSS_sample_gas
./startLSSSampleGasTest
```

This runs the production database in **shadow mode**, with the IOC console in this terminal (type `exit` to stop it). Its settings go to `autosave-test/`, so nothing from the test, including shadow mode, carries over to the production start. Expect, within a few seconds:

- PV names: MFC 15IDC:Alicat1:, O2 15IDC:D1Dmm_calc, CYL none
- start in shadow mode (FORCE_SHADOW): Par:writeEnable set to 0
- a state line, for example `→ IDLE (restart: setpoint is 0 ($4.7))`, or `→ REGULATE` if helium was flowing (then it regulates in shadow and logs shadow mode: would write ...)
- if instead you see `waiting for the Alicat PVs ...`, the Alicat IOC isn't up or its PVs have other names; the controller waits and decides once they connect.

From another terminal:

```
P=15IDC:SampleGas:
caget ${P}Sts:State ${P}Sts:O2 ${P}Sts:Flow ${P}Sts:SetpointRBV \
    ${P}Sts:WriteEnable ${P}Sts:TickAge
caget -S ${P}Sts:Banner
cd <support>/ChemMat/lssSampleGas/iocBoot/iocLSS_sample_gas
tail -f logs/sampleGas_15IDC_*.log
```

`Sts:O2`, `Sts:Flow` and `Sts:SetpointRBV` should match the O2 and Alicat panels you already have; `Sts:WriteEnable` 0 (shadow); `Sts:TickAge` 0 or 1.

5. Screens

Unzip `sampleGas_screens.zip` into a folder next to your other displays and open `sampleGas_simple.bob` (everyday) or `sampleGas_main.bob` (full panel) with the macro `P=15IDC:SampleGas:`. The easiest way is an action button in one of your existing menu displays: "Open display" `sampleGas_simple.bob`, macros `P = 15IDC:SampleGas:`. The displays open each other (Full panel..., Admin..., Deep admin...) and pass `P` on. `docs/ioc/OPERATOR_GUIDE.md` explains every field and button.

In shadow mode both panels show a SHADOW MODE badge.

6. Test sequence

Do it with helium available and the enclosure closed. Stop at any point with **Flow Zero** (the Alicat goes to 0 at once), or Admin → **Shadow (off)** (the controller stops writing and the Alicat keeps its flow).

1. **Shadow, as found.** Watch for a few minutes: readings match, no alarms you can't explain, `Sts:TickAge` stays at 0–1. If it regulates in shadow, the log shows what it would write.
2. **Stop the old timer script.**
3. **Go live:** Admin → **Live (on)** → confirm. Expect IDLE and `writes enabled: Alicat left at <x> SLPM`. The valve doesn't move at the switch.
4. **Take over:**
 - if the box is already purged (O₂ below 10 %), **Resume Flow:** REGULATE from the current flow, no jump;
 - otherwise **Purge:** PRECHECK, then PURGE at 20 SLPM, then the lid check passes within about a minute, then HANDOFF and REGULATE. From air this took about 6–7 minutes on the bench.
5. **Regulate** for as long as you like; the O₂ should settle at the target (0.99 %) with the in-range LED green. Try a target change if you want.
6. **Flow Zero:** the Alicat goes to 0, state `FLOW_ZERO`.
7. **Restart while regulating** (purge or Resume Flow first): type `exit` in the IOC terminal, wait a minute, and run `./startLSSSampleGasTest` again. It comes back regulating in shadow (this script always starts in shadow) from the flow the Alicat held. Admin → Live puts it in IDLE; Resume Flow takes over again without a jump. The *live* restart, where it resumes writing by itself, is the production start: test it after section 7 with `start_ioc 15LSS_sample_gas` while regulating (or `exit` in its `procServ` console and start it again).
8. **Finish:** `exit`. The Alicat keeps its last setpoint; set it by hand or restart the old script if you are not going on to production.

7. Production start

1. Ask for (or add) the `start_ioc` line; check port 20125 is still free in `IOCLIST: 15LSS_sample_gas 20125 1 <support>/ChemMat/lssSampleGas/iocBoot/iocLSS_sample_gas/startLSSSampleGas`
2. Make sure the test IOC is stopped and the old timer script stays off.
3. `start_ioc 15LSS_sample_gas`. This start is live and uses `autosave/`, not the test's settings. With the setpoint at 0 it enters IDLE and waits for Purge or Resume Flow. With helium flowing and the box purged, it resumes regulating at once.
4. Alarm server: load `ioc/screens/sampleGas_alarms.xml` (production prefix). It includes `Sts:Heartbeat` (IOC down) and `Sts:TickAge` (controller stalled). Give the archiver `ioc/screens/archive_pvs.txt`.

`procServ` runs with `--noautorestart`: after a crash, restart it with `start_ioc`. The Alicat holds its last setpoint meanwhile, and the restart resumes without a bump.

8. Updating an installed IOC

```
git -C ~/helium-flow-fb pull
git -C ~/helium-flow-fb log -1 --oneline
rsync -a ~/helium-flow-fb/ioc/lssSampleGas <support>/ChemMat/
cd <support>/ChemMat/lssSampleGas && make 2>&1 | tee build.log
```

rsync without `--delete` keeps what the repo doesn't have: `configure/RELEASE.local`, the `autosave/` and `autosave-test/` settings, and `logs/`. Then restart the IOC to load the new build: `start_ioc 15LSS_sample_gas`, or `exit` in its console and start it again. The restart is bumpless. Without internet, repeat section 1B and unpack over the old copy, which also keeps those files.

Troubleshooting

You see	Meaning / what to do
waiting for the Alicat PVs (...): controller not acting yet (MAJOR)	The Alicat IOC is down, or its PV names differ: <code>caget 15IDC:Alicat1:Setpoint_RBV</code> . Fix the names on Deep admin ("Alicat prefix", Apply) if needed.
not configured: <code>Cfg:... is empty</code>	A PV name field on Deep admin is empty.
MFC gas table is ..., not He (MINOR)	The Alicat's gas table isn't helium; the flow reading is wrong until it is.
flow mismatch: cylinder empty or MFC fault? (MAJOR)	The flow doesn't follow the setpoint: cylinder valve, pressure, regulator.
controller not ticking (red over the banner)	The IOC answers but the controller stopped: restart the IOC.
Screens all magenta, fields show <15IDC:SampleGas:...>	The IOC isn't running, or Phoebe can't reach it (CA address list).